

Cross-validation, the bias-variance decomposition

COURSE MATH 637: Mathematical Techniques in Data Science

WEBSITE <https://jacobcalvert.com/teaching/math637fall2026/>

INSTRUCTOR Jacob Calvert (calvert@udel.edu)

OFFICE HOURS MW 2:45pm–3:45pm, Ewing Hall 509

Announcements & Reminders

- Quiz 3 on linear regression will be on Friday 10/02. It will consist of five multiple choice questions covering material from L9–L11. You will not need to calculate anything.
- We'll finish regression basics this week, then start classification.

Plan for today

- Recap: overfitting and ridge regression 5 min
- Choosing hyperparameters with cross-validation 13 min
- Lasso and elastic net 10 min
- Bias-variance decomposition 13 min
- Ridge accepts bias to reduce variance 8 min

Recall: Advertising data

For each of $n = 200$ markets:

- $x_i \in \mathbb{R}^3$: the budgets for TV, radio and newspaper advertising of a product, in thousands of USD
- y_i : sales of the product, in thousands of units
- 100 training markets and 100 test markets

market	TV	radio	news.	sales
1	230.1	37.8	69.2	22.1
2	44.5	39.3	45.1	10.4
3	17.2	45.9	69.3	9.3
4	151.5	41.3	58.5	18.5
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

James, Witten, Hastie and Tibshirani, *An Introduction to Statistical Learning* (2021)

Recall: Advertising data

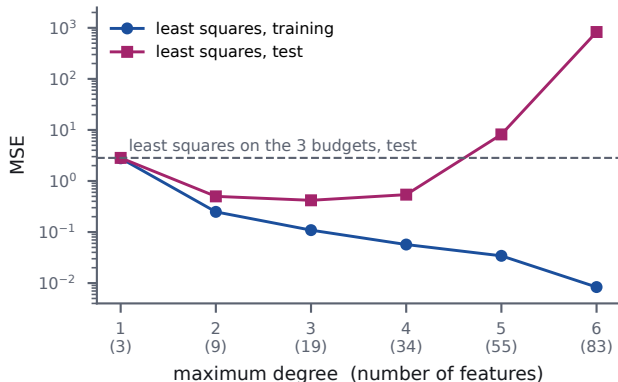
For each of $n = 200$ markets:

- $x_i \in \mathbb{R}^3$: the budgets for TV, radio and newspaper advertising of a product, in thousands of USD
- y_i : sales of the product, in thousands of units
- 100 training markets and 100 test markets
- 83 features: all products of the budgets up to degree 6

market	TV	radio	news.	sales
1	230.1	37.8	69.2	22.1
2	44.5	39.3	45.1	10.4
3	17.2	45.9	69.3	9.3
4	151.5	41.3	58.5	18.5
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

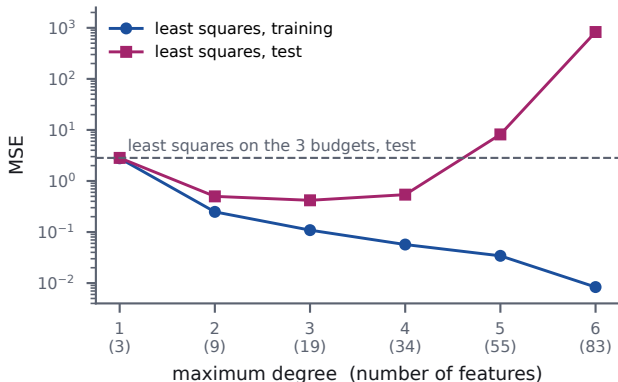
James, Witten, Hastie and Tibshirani, *An Introduction to Statistical Learning* (2021)

Recall: More features, **worse** predictions



- 3 budgets:
 - training MSE = 2.78
 - test MSE = 2.83
 - largest $|\hat{\theta}_j| = 3.90$
- 83 features:
 - training MSE = 0.008
 - test MSE = 828
 - largest $|\hat{\theta}_j| = 126$

Recall: More features, **worse** predictions



- 3 budgets:
 - training MSE = 2.78
 - test MSE = 2.83
 - largest $|\hat{\theta}_j| = 3.90$
- 83 features:
 - training MSE = 0.008
 - test MSE = 828
 - largest $|\hat{\theta}_j| = 126$

Recall: Training MSE $\ll$ test MSE $\Rightarrow$ 83-feature model is **overfit**.

Recall: Ridge regression

Definition (Ridge regression)

With standardized $X \in \mathbb{R}^{n \times p}$ and centered y , for $\lambda > 0$ minimize

$$\text{MSE}_{\lambda}(\theta) = \frac{1}{n} \|y - X\theta\|^2 + \lambda \|\theta\|^2.$$

	training MSE	test MSE	largest $ \hat{\theta}_j $
least squares, 3 budgets	2.78	2.83	3.90
least squares, 83 features	0.008	828	126
ridge, 83 features, $\lambda = 10^{-2}$	0.049	0.651	2.73

Recall: Ridge regression

Definition (Ridge regression)

With standardized $X \in \mathbb{R}^{n \times p}$ and centered y , for $\lambda > 0$ minimize

$$\text{MSE}_{\lambda}(\theta) = \frac{1}{n} \|y - X\theta\|^2 + \lambda \|\theta\|^2.$$

	training MSE	test MSE	largest $ \hat{\theta}_j $
least squares, 3 budgets	2.78	2.83	3.90
least squares, 83 features	0.008	828	126
ridge, 83 features, $\lambda = 10^{-2}$	0.049	0.651	2.73

Problem

We found $\lambda = 10^{-2}$ by looking at the **test** MSE. Poor methodology!

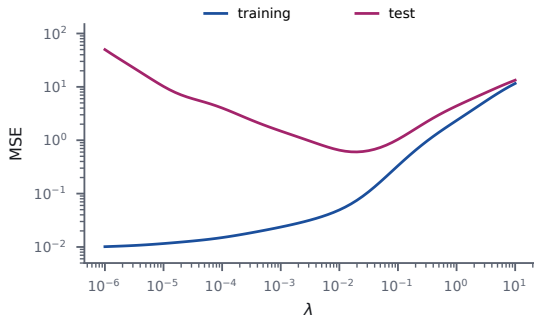
Plan for today

- Recap: overfitting and ridge regression 5 min
- Choosing hyperparameters with cross-validation 13 min
- Lasso and elastic net 10 min
- Bias-variance decomposition 13 min
- Ridge accepts bias to reduce variance 8 min

Definition: Hyperparameter

Definition

A **hyperparameter**, like λ or the **degree** of polynomial features, *specifies the fitting procedure*. It is not found by minimizing training MSE.



Idea: Introduce a validation set



Watch out: Choosing λ , features or preprocessing with the test set can make its MSE optimistic.

Idea: Introduce a validation set



Watch out: Choosing λ , features or preprocessing with the test set can make its MSE optimistic.

Note: Using plots of test MSE vs. degree and λ to choose the hyperparameters is poor methodology.

Idea: Introduce a validation set



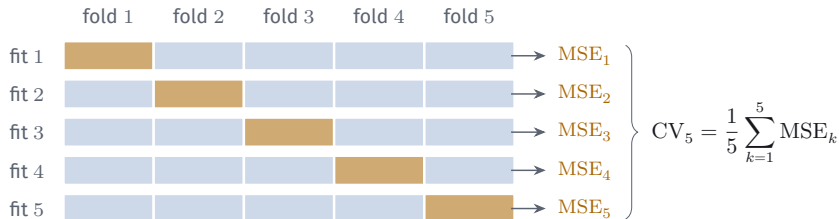
Watch out: Choosing λ , features or preprocessing with the test set can make its MSE optimistic.

Note: Using plots of test MSE vs. degree and λ to choose the hyperparameters is poor methodology.

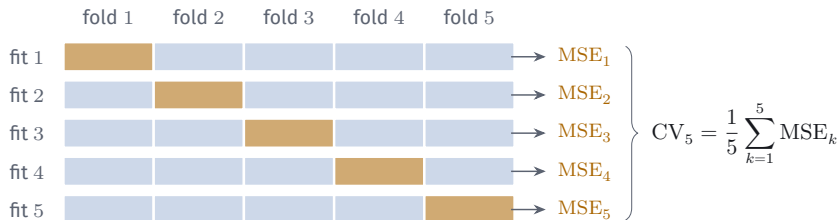
Problem

The validation set leaves less data for training!

Definition: K -fold cross-validation



Definition: K -fold cross-validation



Definition (K -fold cross-validation)

Partition the training examples into “folds” $I_1, \dots, I_K$. For $\lambda > 0$, calculate

$$CV_K(\lambda) = \frac{1}{K} \sum_{k=1}^K MSE_k,$$

where MSE_k is the MSE on fold I_k of the model fitted on **all but** I_k .

Choosing the penalty λ with K -fold CV

- Approach:

Choosing the penalty λ with K -fold CV

- Approach:
 1. Partition the training data into K folds of equal size.

Choosing the penalty λ with K -fold CV

- Approach:
 1. Partition the training data into K folds of equal size.
 2. For each candidate λ , compute $CV_K(\lambda)$ using this partition.

Choosing the penalty λ with K -fold CV

- Approach:
 1. Partition the training data into K folds of equal size.
 2. For each candidate λ , compute $CV_K(\lambda)$ using this partition.
 3. Choose the λ that minimizes $CV_K(\lambda)$.

Choosing the penalty λ with K -fold CV

- Approach:
 1. Partition the training data into K folds of equal size.
 2. For each candidate λ , compute $CV_K(\lambda)$ using this partition.
 3. Choose the λ that minimizes $CV_K(\lambda)$.
 4. Fit the model on the **entire training set** with this value of λ .

Choosing the penalty λ with K -fold CV

- Approach:
 1. Partition the training data into K folds of equal size.
 2. For each candidate λ , compute $CV_K(\lambda)$ using this partition.
 3. Choose the λ that minimizes $CV_K(\lambda)$.
 4. Fit the model on the **entire training set** with this value of λ .
- **Note:** Do the preprocessing (scalers, features) separately for each fold's training data!

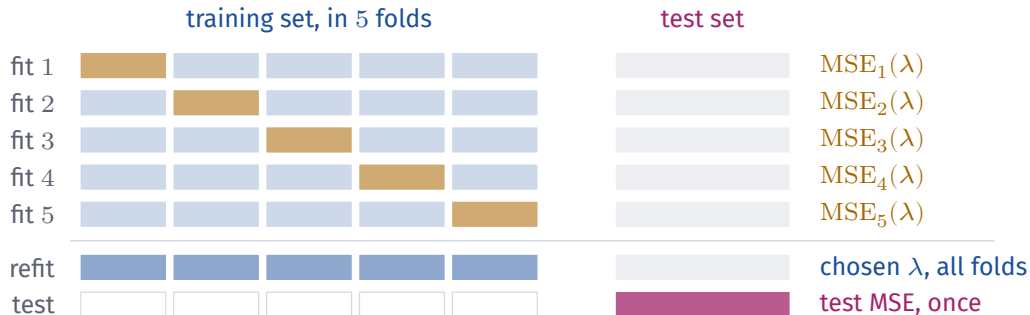
Choosing the penalty λ with K -fold CV

- Approach:
 1. Partition the training data into K folds of equal size.
 2. For each candidate λ , compute $CV_K(\lambda)$ using this partition.
 3. Choose the λ that minimizes $CV_K(\lambda)$.
 4. Fit the model on the **entire training set** with this value of λ .
- **Note:** Do the preprocessing (scalers, features) separately for each fold's training data!
- **Note:** There are tradeoffs in choosing K . Common choices are $K \in \{4, 5, 10\}$.

Choosing the penalty λ with K -fold CV

- Approach:
 1. Partition the training data into K folds of equal size.
 2. For each candidate λ , compute $CV_K(\lambda)$ using this partition.
 3. Choose the λ that minimizes $CV_K(\lambda)$.
 4. Fit the model on the **entire training set** with this value of λ .
- **Note:** Do the preprocessing (scalers, features) separately for each fold's training data!
- **Note:** There are tradeoffs in choosing K . Common choices are $K \in \{4, 5, 10\}$.
- **Note:** The special case of $K = n_{\text{train}}$ is called “leave-one-out cross-validation” (LOOCV).

Example: Train-test split with 5-fold CV



In each fit and in the refit, scalars, features and ridge are fit to the **blue** blocks only.

Python: Define the preprocessing–fitting pipeline

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge
pipe = make_pipeline(
    StandardScaler(),
    PolynomialFeatures(degree=6, include_bias=False),
    StandardScaler(),
    Ridge(solver="svd"))
```

Python: Do cross-validation *around* entire pipeline

```
from sklearn.model_selection import GridSearchCV

search = GridSearchCV(
    pipe, {"ridge__alpha": np.logspace(-6, 3, 91)},
    scoring="neg_mean_squared_error", cv=5, refit=True)
search.fit(Z_train, y_train)
```

- "ridge__alpha" is the alpha of the pipeline's Ridge step

Python: Do cross-validation *around* entire pipeline

```
from sklearn.model_selection import GridSearchCV

search = GridSearchCV(
    pipe, {"ridge__alpha": np.logspace(-6, 3, 91)},
    scoring="neg_mean_squared_error", cv=5, refit=True)
search.fit(Z_train, y_train)
```

- "ridge__alpha" is the alpha of the pipeline's Ridge step
- same 5 folds used for every α , and every step of pipeline is refit on each fold's training rows

Python: Do cross-validation *around* entire pipeline

```
from sklearn.model_selection import GridSearchCV

search = GridSearchCV(
    pipe, {"ridge__alpha": np.logspace(-6, 3, 91)},
    scoring="neg_mean_squared_error", cv=5, refit=True)
search.fit(Z_train, y_train)
```

- "ridge__alpha" is the alpha of the pipeline's Ridge step
- same 5 folds used for every α , and every step of pipeline is refit on each fold's training rows
- refit=True: refit on *all* training rows with best α

Python: Do cross-validation *around* entire pipeline

```
from sklearn.model_selection import GridSearchCV

search = GridSearchCV(
    pipe, {"ridge__alpha": np.logspace(-6, 3, 91)},
    scoring="neg_mean_squared_error", cv=5, refit=True)
search.fit(Z_train, y_train)
```

- "ridge__alpha" is the `alpha` of the pipeline's `Ridge` step
- same 5 folds used for every α , and every step of pipeline is refit on each fold's training rows
- `refit=True`: refit on *all* training rows with best α
- "neg_mean_squared_error" because it *maximizes* a score

Python: Evaluate the chosen model

```
from sklearn.metrics import mean_squared_error

best_alpha = search.best_params_["ridge__alpha"]
selected_cv_mse = -search.best_score_
y_pred = search.predict(Z_test)
test_mse = mean_squared_error(y_test, y_pred)
```

- `search.predict` uses the pipeline refit on all training rows, with the *training* scalers used to standardize the *test* data

Python: Evaluate the chosen model

```
from sklearn.metrics import mean_squared_error

best_alpha = search.best_params_["ridge__alpha"]
selected_cv_mse = -search.best_score_
y_pred = search.predict(Z_test)
test_mse = mean_squared_error(y_test, y_pred)
```

- `search.predict` uses the pipeline refit on all training rows, with the *training* scalers used to standardize the *test* data
- test set appears only in the last two lines!

Example: 5-fold CV on the advertising data

- 5-fold CV chooses $\alpha = 10$.
- Recall that $\alpha = m\lambda$, where m = no. of examples used to fit the model.
- The final fit on all the training data with the best α therefore corresponds to $\lambda = 10/100 = 0.1$.
- The cross-validation MSE with the chosen α is 1.22
- Test set MSE = 1.04, against 828 for least squares on the 83 features

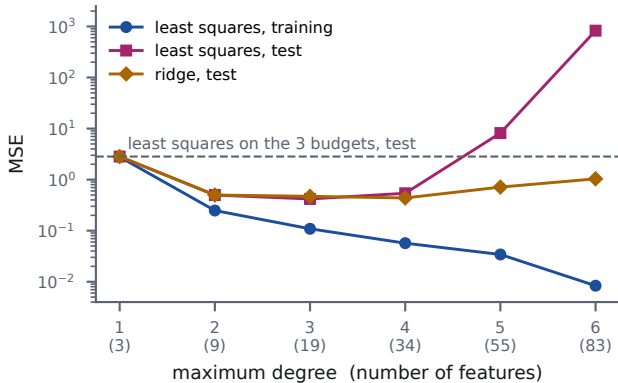
Example: 5-fold CV on the advertising data

- 5-fold CV chooses $\alpha = 10$.
- Recall that $\alpha = m\lambda$, where m = no. of examples used to fit the model.
- The final fit on all the training data with the best α therefore corresponds to $\lambda = 10/100 = 0.1$.
- The cross-validation MSE with the chosen α is 1.22
- Test set MSE = 1.04, against 828 for least squares on the 83 features

Check

Recall that we had a test set MSE of 0.65 with $\lambda = 10^{-2}$. Why does it make sense that CV did not choose this λ ?

Ridge at **every** degree



Median test MSE, 100 splits

least squares, 3	2.99
least squares, degree 3	0.40
least squares, 83	1195
ridge, 83	0.58

A comparison looking back; λ chosen by 5-fold CV in each split.

Plan for today

- Recap: overfitting and ridge regression 5 min
- Choosing hyperparameters with cross-validation 13 min
- Lasso and elastic net 10 min
- Bias-variance decomposition 13 min
- Ridge accepts bias to reduce variance 8 min

Definition: Lasso and elastic net

Recall that ridge regression minimizes $\text{MSE}(\theta) + \lambda \|\theta\|^2$.

Definition (Lasso; Santosa & Symes, 1986 and Tibshirani, 1996)

Minimize $\text{MSE}(\theta) + \lambda \sum_j |\theta_j|$.

Definition (Elastic net; Zou & Hastie, 2005)

Minimize $\text{MSE}(\theta) + \lambda_1 \sum_j |\theta_j| + \lambda_2 \|\theta\|^2$.

- Lasso **can set coefficients exactly to 0**; ridge generally does not

F. Santosa and W. Symes, SIAM J. Sci. Stat. Comput. **7** (1986)
R. Tibshirani, J. R. Stat. Soc. B **58** (1996)
H. Zou and T. Hastie, J. R. Stat. Soc. B **67** (2005)

Definition: Lasso and elastic net

Recall that ridge regression minimizes $\text{MSE}(\theta) + \lambda \|\theta\|^2$.

Definition (Lasso; Santosa & Symes, 1986 and Tibshirani, 1996)

Minimize $\text{MSE}(\theta) + \lambda \sum_j |\theta_j|$.

Definition (Elastic net; Zou & Hastie, 2005)

Minimize $\text{MSE}(\theta) + \lambda_1 \sum_j |\theta_j| + \lambda_2 \|\theta\|^2$.

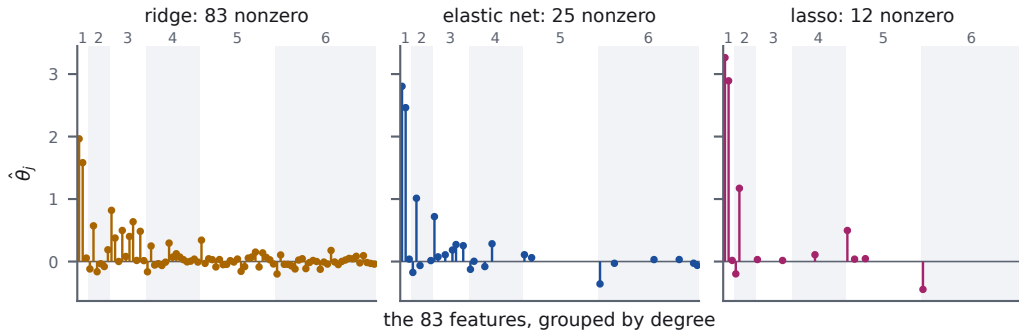
- Lasso **can set coefficients exactly to 0**; ridge generally does not
- usually solved numerically by coordinate descent

F. Santosa and W. Symes, SIAM J. Sci. Stat. Comput. **7** (1986)

R. Tibshirani, J. R. Stat. Soc. B **58** (1996)

H. Zou and T. Hastie, J. R. Stat. Soc. B **67** (2005)

Application: Three penalties



Coefficients on the 83 features, each penalty chosen by 5-fold cross-validation on the 100 training markets, with the pipeline refit in each fold.

Application: Three penalties

	features kept	test MSE	median, 100 splits
least squares, 3 budgets	3	2.83	2.99
least squares	83	828	1195
ridge	83	1.04	0.58
elastic net, $r = 0.5$	25	0.37	0.30
lasso	12	0.26	0.24

- on these data the lasso predicts best; that is not a general ranking

Application: Three penalties

	features kept	test MSE	median, 100 splits
least squares, 3 budgets	3	2.83	2.99
least squares	83	828	1195
ridge	83	1.04	0.58
elastic net, $r = 0.5$	25	0.37	0.30
lasso	12	0.26	0.24

- on these data the lasso predicts best; that is not a general ranking
- the lasso keeps TV, radio and TV·radio in every split

Application: Three penalties

	features kept	test MSE	median, 100 splits
least squares, 3 budgets	3	2.83	2.99
least squares	83	828	1195
ridge	83	1.04	0.58
elastic net, $r = 0.5$	25	0.37	0.30
lasso	12	0.26	0.24

- on these data the lasso predicts best; that is not a general ranking
- the lasso keeps TV, radio and TV·radio in every split

Application: Three penalties

	features kept	test MSE	median, 100 splits
least squares, 3 budgets	3	2.83	2.99
least squares	83	828	1195
ridge	83	1.04	0.58
elastic net, $r = 0.5$	25	0.37	0.30
lasso	12	0.26	0.24

- on these data the lasso predicts best; that is not a general ranking
- the lasso keeps TV, radio and TV·radio in every split

Watch out: The kept features are not “the true model”: they change from split to split.

Which penalty?

	ridge	elastic net	lasso
coefficients	shrinks them collectively; generally no exact zeros	can set some exactly to 0	can set coefficients exactly to 0
correlated features	share the weight	tends to keep or drop them together	selection among them can be unstable
computing	one linear system	usually numerical; two hyperparameters	usually numerical, often coordinate descent
suits	many small effects	many correlated features, some irrelevant	few features matter; want to read the model

Example: Lasso using scikit-learn

```
from sklearn.linear_model import Lasso
lasso_pipe = make_pipeline(
    StandardScaler(),
    PolynomialFeatures(degree=6, include_bias=False),
    StandardScaler(),
    Lasso(max_iter=100_000))
lasso_search = GridSearchCV(
    lasso_pipe, {"lasso__alpha": np.logspace(-3, 1, 60)},
    scoring="neg_mean_squared_error", cv=5)
lasso_search.fit(Z_train, y_train)
```

- **Lasso** minimizes $\frac{1}{2n} \|y - X\theta\|^2 + \alpha \sum_j |\theta_j|$, so $\alpha = \lambda/2$

Example: Lasso using scikit-learn

```
from sklearn.linear_model import Lasso
lasso_pipe = make_pipeline(
    StandardScaler(),
    PolynomialFeatures(degree=6, include_bias=False),
    StandardScaler(),
    Lasso(max_iter=100_000))
lasso_search = GridSearchCV(
    lasso_pipe, {"lasso__alpha": np.logspace(-3, 1, 60)},
    scoring="neg_mean_squared_error", cv=5)
lasso_search.fit(Z_train, y_train)
```

- `Lasso` minimizes $\frac{1}{2n} \|y - X\theta\|^2 + \alpha \sum_j |\theta_j|$, so $\alpha = \lambda/2$
- `max_iter` raises the cap on iterations, not the tolerance

Example: ElasticNet using scikit-learn

```
from sklearn.linear_model import ElasticNet
enet_pipe = make_pipeline(
    StandardScaler(),
    PolynomialFeatures(degree=6, include_bias=False),
    StandardScaler(),
    ElasticNet(max_iter=100_000))
grid = {"elasticnet__alpha": np.logspace(-3, 1, 60),
        "elasticnet__l1_ratio": [0.1, 0.5, 0.9, 1.0]}
enet_search = GridSearchCV(enet_pipe, grid,
                           scoring="neg_mean_squared_error", cv=5)
enet_search.fit(Z_train, y_train)
```

- $\lambda_1 = 2\alpha r$ and $\lambda_2 = \alpha(1 - r)$, with $r = \text{l1_ratio}$

Plan for today

- Recap: overfitting and ridge regression 5 min
- Choosing hyperparameters with cross-validation 13 min
- Lasso and elastic net 10 min
- Bias-variance decomposition 13 min
- Ridge accepts bias to reduce variance 8 min

Summary of rest of lecture

- **Idea:** we can write the squared prediction error on an example as
$$\text{squared prediction error} = \text{noise} + \text{bias}^2 + \text{variance}.$$

Summary of rest of lecture

- **Idea:** we can write the squared prediction error on an example as
$$\text{squared prediction error} = \text{noise} + \text{bias}^2 + \text{variance}.$$
- We can't get rid of **noise**, but we can trade-off **bias²** and **variance**.

Summary of rest of lecture

- **Idea:** we can write the squared prediction error on an example as

squared prediction error = noise + bias² + variance.

- We can't get rid of noise, but we can trade-off bias² and variance.
- High bias² = underfitting; high variance = overfitting.

Summary of rest of lecture

- **Idea:** we can write the squared prediction error on an example as

$$\text{squared prediction error} = \text{noise} + \text{bias}^2 + \text{variance}.$$

- We can't get rid of **noise**, but we can trade-off **bias²** and **variance**.
- High **bias²** = underfitting; high **variance** = overfitting.
- Least squares is **unbiased**, but can have high **variance**.

Summary of rest of lecture

- **Idea:** we can write the squared prediction error on an example as

$$\text{squared prediction error} = \text{noise} + \text{bias}^2 + \text{variance}.$$

- We can't get rid of **noise**, but we can trade-off **bias²** and **variance**.
- High **bias²** = underfitting; high **variance** = overfitting.
- Least squares is **unbiased**, but can have high **variance**.
- **Insight:** ridge accepts some **bias** to greatly decrease **variance**!

Summary of rest of lecture

- **Idea:** we can write the squared prediction error on an example as

$$\text{squared prediction error} = \text{noise} + \text{bias}^2 + \text{variance}.$$

- We can't get rid of **noise**, but we can trade-off **bias²** and **variance**.
- High **bias²** = underfitting; high **variance** = overfitting.
- Least squares is **unbiased**, but can have high **variance**.
- **Insight:** ridge accepts some **bias** to greatly decrease **variance**!
- We explore these ideas using the Gaussian linear model from L10.

Recall: Gaussian linear model

Definition (Gaussian linear model)

Given a parameter vector $\theta \in \mathbb{R}^p$ and noise variance σ^2 , we assume that

$$y = f(x) + \varepsilon,$$

where $x \in \mathbb{R}^p$, $f(x) = \theta^\top x$, and ε is $\mathcal{N}(0, \sigma^2)$ noise.

- In particular, if $x_1, \dots, x_n \in \mathbb{R}^p$ are training examples, we assume that

$$y_i = f(x_i) + \varepsilon_i,$$

where the ε_i are independent.

Recall: Gaussian linear model

Definition (Gaussian linear model)

Given a parameter vector $\theta \in \mathbb{R}^p$ and noise variance σ^2 , we assume that

$$y = f(x) + \varepsilon,$$

where $x \in \mathbb{R}^p$, $f(x) = \theta^\top x$, and ε is $\mathcal{N}(0, \sigma^2)$ noise.

- In particular, if $x_1, \dots, x_n \in \mathbb{R}^p$ are training examples, we assume that

$$y_i = f(x_i) + \varepsilon_i,$$

where the ε_i are independent.

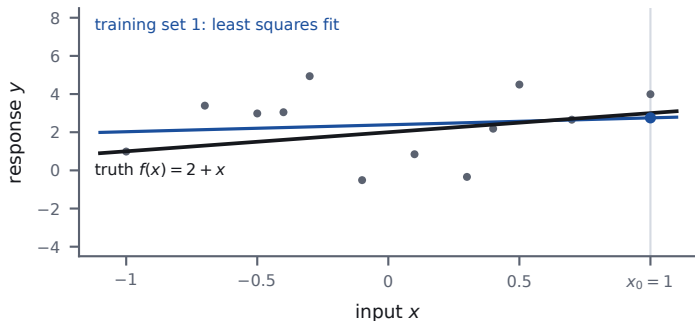
- Even when f and x_i are fixed, noise gives different training sets

$$D = \{(x_1, y_1), \dots, (x_n, y_n)\},$$

hence different fits $\hat{f}_D$.

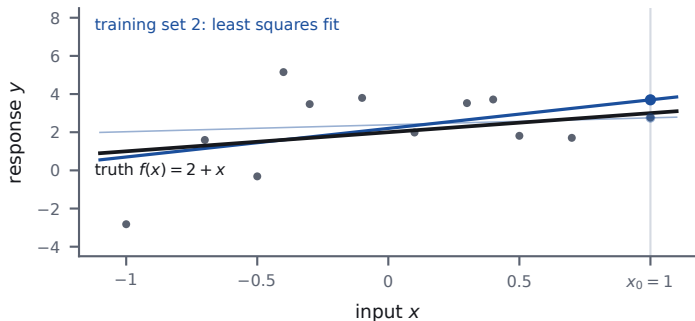
Example: Different training sets and fits

Setup: $f(x) = 2 + x$, fixed $x_1, \dots, x_{12}$, noise w/ variance $\sigma = 2$, least squares fit.



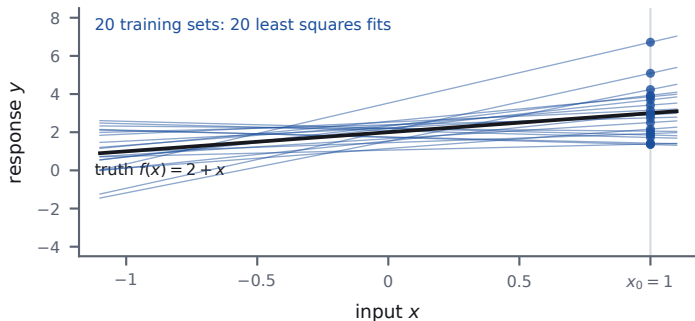
Example: Different training sets and fits

Setup: $f(x) = 2 + x$, fixed $x_1, \dots, x_{12}$, noise w/ variance $\sigma = 2$, least squares fit.



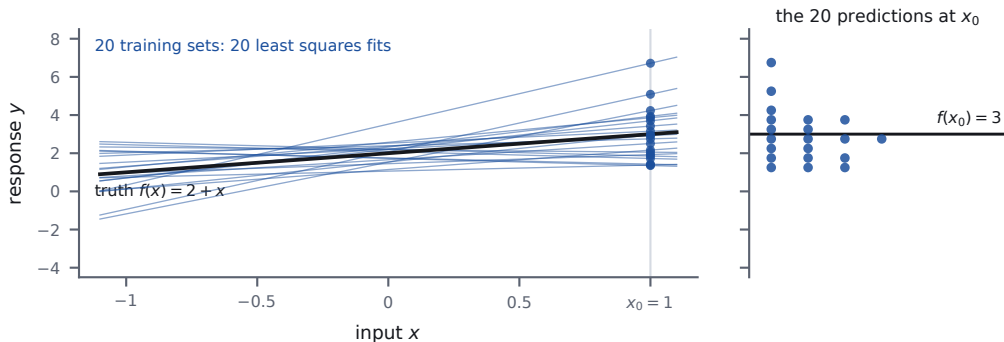
Example: Different training sets and fits

Setup: $f(x) = 2 + x$, fixed $x_1, \dots, x_{12}$, noise w/ variance $\sigma = 2$, least squares fit.



Example: Different training sets and fits

Setup: $f(x) = 2 + x$, fixed $x_1, \dots, x_{12}$, noise w/ variance $\sigma = 2$, least squares fit.



Same inputs and “true” f , different response noise and fitted predictions.

Two sources of randomness

- The training set D varies through the training noise $\varepsilon_1, \dots, \varepsilon_n$.

Two sources of randomness

- The training set D varies through the training noise $\varepsilon_1, \dots, \varepsilon_n$.
- At a fixed input x_0 , each D gives a fitted prediction $\hat{f}_D(x_0)$.

Two sources of randomness

- The training set D varies through the training noise $\varepsilon_1, \dots, \varepsilon_n$.
- At a fixed input x_0 , each D gives a fitted prediction $\hat{f}_D(x_0)$.
- Averaging over the training noise gives

$$m(x_0) = \mathbb{E}_D[\hat{f}_D(x_0)].$$

Two sources of randomness

- The training set D varies through the training noise $\varepsilon_1, \dots, \varepsilon_n$.
- At a fixed input x_0 , each D gives a fitted prediction $\hat{f}_D(x_0)$.
- Averaging over the training noise gives

$$m(x_0) = \mathbb{E}_D[\hat{f}_D(x_0)].$$

- The new response's noise ε_0 is a separate, independent source of randomness.

Two sources of randomness

- The training set D varies through the training noise $\varepsilon_1, \dots, \varepsilon_n$.
- At a fixed input x_0 , each D gives a fitted prediction $\hat{f}_D(x_0)$.
- Averaging over the training noise gives

$$m(x_0) = \mathbb{E}_D[\hat{f}_D(x_0)].$$

- The new response's noise ε_0 is a separate, independent source of randomness.

Two sources of randomness

- The training set D varies through the training noise $\varepsilon_1, \dots, \varepsilon_n$.
- At a fixed input x_0 , each D gives a fitted prediction $\hat{f}_D(x_0)$.
- Averaging over the training noise gives

$$m(x_0) = \mathbb{E}_D[\hat{f}_D(x_0)].$$

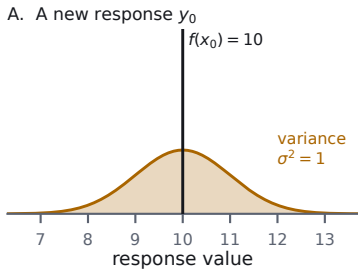
- The new response's noise ε_0 is a separate, independent source of randomness.

Question

How large is $\mathbb{E}[(y_0 - \hat{f}_D(x_0))^2]$, averaging over both D and ε_0 ?

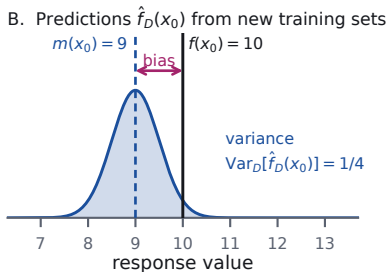
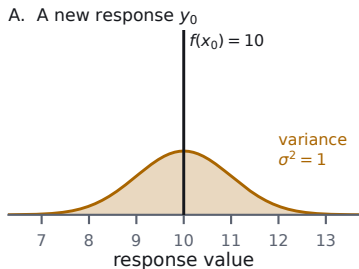
Three contributions to prediction error

Illustrative distributions at one fixed input x_0 :



Three contributions to prediction error

Illustrative distributions at one fixed input x_0 :



$$\mathbb{E}[(y_0 - \hat{f}_D(x_0))^2] = \underbrace{\sigma^2}_{\text{noise}} + \underbrace{[m(x_0) - f(x_0)]^2}_{\text{bias}^2} + \underbrace{\text{Var}_D[\hat{f}_D(x_0)]}_{\text{variance}}$$

Theorem: Bias–variance decomposition

Theorem

If $\mathbb{E}\varepsilon_0 = 0$, $\text{Var}\varepsilon_0 = \sigma^2$ and ε_0 is independent of D , then

$$\mathbb{E}[(y_0 - \hat{f}_D(x_0))^2] = \underbrace{\sigma^2}_{\text{noise}} + \underbrace{[m(x_0) - f(x_0)]^2}_{\text{bias}^2} + \underbrace{\text{Var}_D[\hat{f}_D(x_0)]}_{\text{variance}}.$$

- **noise**: variation of a new response, given its inputs

Theorem: Bias–variance decomposition

Theorem

If $\mathbb{E}\varepsilon_0 = 0$, $\text{Var}\varepsilon_0 = \sigma^2$ and ε_0 is independent of D , then

$$\mathbb{E}[(y_0 - \hat{f}_D(x_0))^2] = \underbrace{\sigma^2}_{\text{noise}} + \underbrace{[m(x_0) - f(x_0)]^2}_{\text{bias}^2} + \underbrace{\text{Var}_D[\hat{f}_D(x_0)]}_{\text{variance}}.$$

- **noise**: variation of a new response, given its inputs
- **bias**: how far the mean fitted prediction is from the truth

Theorem: Bias–variance decomposition

Theorem

If $\mathbb{E}\varepsilon_0 = 0$, $\text{Var}\varepsilon_0 = \sigma^2$ and ε_0 is independent of D , then

$$\mathbb{E}[(y_0 - \hat{f}_D(x_0))^2] = \underbrace{\sigma^2}_{\text{noise}} + \underbrace{[m(x_0) - f(x_0)]^2}_{\text{bias}^2} + \underbrace{\text{Var}_D[\hat{f}_D(x_0)]}_{\text{variance}}.$$

- **noise**: variation of a new response, given its inputs
- **bias**: how far the mean fitted prediction is from the truth
- **variance**: variation of the fitted predictions across training sets

Theorem: Bias–variance decomposition

Theorem

If $\mathbb{E}\varepsilon_0 = 0$, $\text{Var}\varepsilon_0 = \sigma^2$ and ε_0 is independent of D , then

$$\mathbb{E}[(y_0 - \hat{f}_D(x_0))^2] = \underbrace{\sigma^2}_{\text{noise}} + \underbrace{[m(x_0) - f(x_0)]^2}_{\text{bias}^2} + \underbrace{\text{Var}_D[\hat{f}_D(x_0)]}_{\text{variance}}.$$

- **noise**: variation of a new response, given its inputs
- **bias**: how far the mean fitted prediction is from the truth
- **variance**: variation of the fitted predictions across training sets
- f need not be linear, and the noise need not be Gaussian

Why the three terms add

Write $F = \hat{f}_D(x_0)$, $f_0 = f(x_0)$ and $m = \mathbb{E}_D[F]$.

- **Step 1.** Separate ε_0 from the rest:

$$\mathbb{E}[(y_0 - F)^2] = \mathbb{E}[(\varepsilon_0 + f_0 - F)^2] = \sigma^2 + \mathbb{E}[(F - f_0)^2],$$

because $\mathbb{E}[\varepsilon_0] = 0$ and ε_0 is independent of F

Why the three terms add

Write $F = \hat{f}_D(x_0)$, $f_0 = f(x_0)$ and $m = \mathbb{E}_D[F]$.

- **Step 1.** Separate ε_0 from the rest:

$$\mathbb{E}[(y_0 - F)^2] = \mathbb{E}[(\varepsilon_0 + f_0 - F)^2] = \sigma^2 + \mathbb{E}[(F - f_0)^2],$$

because $\mathbb{E}[\varepsilon_0] = 0$ and ε_0 is independent of F

- **Step 2.** Center the fitted prediction F around its mean m :

$$\mathbb{E}[(F - f_0)^2] = \mathbb{E}[((F - m) + (m - f_0))^2] = \text{Var}(F) + (m - f_0)^2,$$

because $\mathbb{E}[F - m] = 0$ and $m - f_0$ is a constant

Plan for today

- Recap: overfitting and ridge regression 5 min
- Choosing hyperparameters with cross-validation 13 min
- Lasso and elastic net 10 min
- Bias–variance decomposition 13 min
- Ridge accepts bias to reduce variance 8 min

Observation: Least squares is unbiased

If the linear model is correct, averaging the least squares estimates over repeated training sets recovers the true coefficients.

- assume X fixed, of full column rank; $y = X\theta + \varepsilon$; $\mathbb{E}[\varepsilon] = 0$

Observation: Least squares is unbiased

If the linear model is correct, averaging the least squares estimates over repeated training sets recovers the true coefficients.

- assume X fixed, of full column rank; $y = X\theta + \varepsilon$; $\mathbb{E}[\varepsilon] = 0$
- substitute $y = X\theta + \varepsilon$ into solution of least squares:

$$\hat{\theta}_{\text{LS}} = (X^T X)^{-1} X^T y = \theta + (X^T X)^{-1} X^T \varepsilon, \quad \text{so} \quad \boxed{\mathbb{E}[\hat{\theta}_{\text{LS}}] = \theta}$$

Observation: Least squares is unbiased

If the linear model is correct, averaging the least squares estimates over repeated training sets recovers the true coefficients.

- assume X fixed, of full column rank; $y = X\theta + \varepsilon$; $\mathbb{E}[\varepsilon] = 0$
- substitute $y = X\theta + \varepsilon$ into solution of least squares:

$$\hat{\theta}_{\text{LS}} = (X^T X)^{-1} X^T y = \theta + (X^T X)^{-1} X^T \varepsilon, \quad \text{so} \quad \boxed{\mathbb{E}[\hat{\theta}_{\text{LS}}] = \theta}$$

- this implies that $\text{bias}^2 = 0$ because

$$\text{bias} = m(x_0) - f(x_0) = \mathbb{E}[\hat{\theta}_{\text{LS}}^T x_0] - \theta^T x_0 = \mathbb{E}[\hat{\theta}_{\text{LS}}]^T x_0 - \theta^T x_0 = 0.$$

Observation: Least squares is unbiased

If the linear model is correct, averaging the least squares estimates over repeated training sets recovers the true coefficients.

- assume X fixed, of full column rank; $y = X\theta + \varepsilon$; $\mathbb{E}[\varepsilon] = 0$
- substitute $y = X\theta + \varepsilon$ into solution of least squares:

$$\hat{\theta}_{\text{LS}} = (X^T X)^{-1} X^T y = \theta + (X^T X)^{-1} X^T \varepsilon, \quad \text{so} \quad \boxed{\mathbb{E}[\hat{\theta}_{\text{LS}}] = \theta}$$

- this implies that $\text{bias}^2 = 0$ because

$$\text{bias} = m(x_0) - f(x_0) = \mathbb{E}[\hat{\theta}_{\text{LS}}^T x_0] - \theta^T x_0 = \mathbb{E}[\hat{\theta}_{\text{LS}}]^T x_0 - \theta^T x_0 = 0.$$

Observation: Least squares is unbiased

If the linear model is correct, averaging the least squares estimates over repeated training sets recovers the true coefficients.

- assume X fixed, of full column rank; $y = X\theta + \varepsilon$; $\mathbb{E}[\varepsilon] = 0$
- substitute $y = X\theta + \varepsilon$ into solution of least squares:

$$\hat{\theta}_{\text{LS}} = (X^T X)^{-1} X^T y = \theta + (X^T X)^{-1} X^T \varepsilon, \quad \text{so} \quad \boxed{\mathbb{E}[\hat{\theta}_{\text{LS}}] = \theta}$$

- this implies that $\text{bias}^2 = 0$ because

$$\text{bias} = m(x_0) - f(x_0) = \mathbb{E}[\hat{\theta}_{\text{LS}}^T x_0] - \theta^T x_0 = \mathbb{E}[\hat{\theta}_{\text{LS}}]^T x_0 - \theta^T x_0 = 0.$$

Note: One fit can still be far from the truth. Unbiasedness is about the average over training sets.

Key idea: “Shrink” $\hat{\theta}$

Multiply $\hat{\theta}_{\text{LS}}$ by a factor $a \in [0, 1]$ to get $\hat{\theta}_a = a \hat{\theta}_{\text{LS}}$. What happens?

- Denoting $\text{Var } \hat{\theta}_{\text{LS}}$ by v , since $\mathbb{E}[\hat{\theta}_{\text{LS}}] = \theta$, bias and variance become

$$\mathbb{E}[\hat{\theta}_a] - \theta = (a - 1)\theta \quad \text{and} \quad \text{Var } \hat{\theta}_a = a^2 v.$$

Key idea: “Shrink” $\hat{\theta}$

Multiply $\hat{\theta}_{\text{LS}}$ by a factor $a \in [0, 1]$ to get $\hat{\theta}_a = a \hat{\theta}_{\text{LS}}$. What happens?

- Denoting $\text{Var } \hat{\theta}_{\text{LS}}$ by v , since $\mathbb{E}[\hat{\theta}_{\text{LS}}] = \theta$, bias and variance become

$$\mathbb{E}[\hat{\theta}_a] - \theta = (a - 1)\theta \quad \text{and} \quad \text{Var } \hat{\theta}_a = a^2 v.$$

- Step 2 of decomposition proof shows that

$$\boxed{\mathbb{E}[(\hat{\theta}_a - \theta)^2] = (1 - a)^2 \theta^2 + a^2 v.}$$

Key idea: “Shrink” $\hat{\theta}$

Multiply $\hat{\theta}_{\text{LS}}$ by a factor $a \in [0, 1]$ to get $\hat{\theta}_a = a \hat{\theta}_{\text{LS}}$. What happens?

- Denoting $\text{Var } \hat{\theta}_{\text{LS}}$ by v , since $\mathbb{E}[\hat{\theta}_{\text{LS}}] = \theta$, bias and variance become

$$\mathbb{E}[\hat{\theta}_a] - \theta = (a - 1)\theta \quad \text{and} \quad \text{Var } \hat{\theta}_a = a^2 v.$$

- Step 2 of decomposition proof shows that

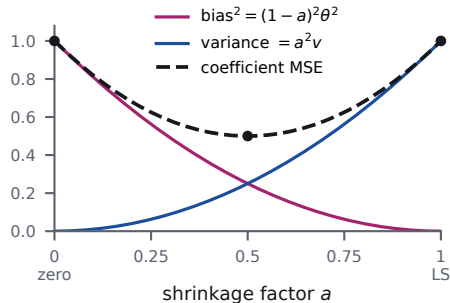
$$\mathbb{E}[(\hat{\theta}_a - \theta)^2] = (1 - a)^2 \theta^2 + a^2 v.$$

- It turns out that ridge does exactly this, with $a = \frac{\sum_i x_i^2}{\sum_i x_i^2 + n\lambda}$!

Example: $\theta = 1$ and $v = 1$

From previous slide: $\mathbb{E}[(\hat{\theta}_a - \theta)^2] = (1 - a)^2\theta^2 + a^2v$.

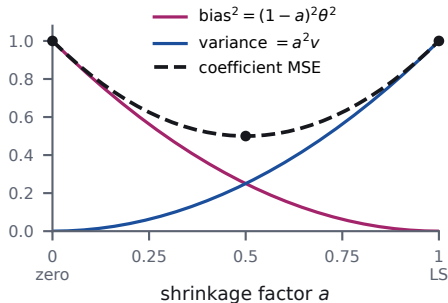
	bias ²	variance	coef. MSE
$a = 1$ (LS)	0	1	1
$a = \frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{2}$
$a = 0$	1	0	1



Example: $\theta = 1$ and $v = 1$

From previous slide: $\mathbb{E}[(\hat{\theta}_a - \theta)^2] = (1 - a)^2\theta^2 + a^2v$.

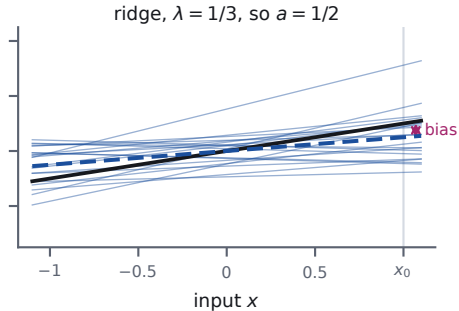
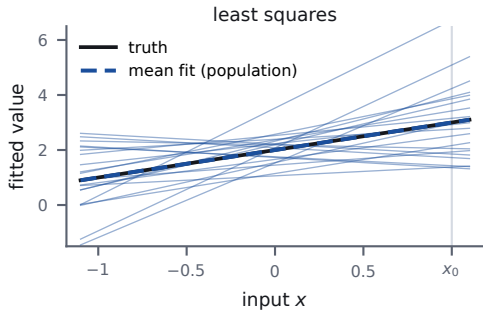
	bias ²	variance	coef. MSE
$a = 1$ (LS)	0	1	1
$a = \frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{2}$
$a = 0$	1	0	1



Note: Some shrinkage halves the error in the coefficient; too much raises it again! A prediction's error also contains the noise σ^2 .

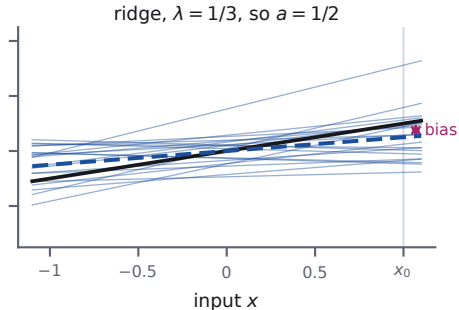
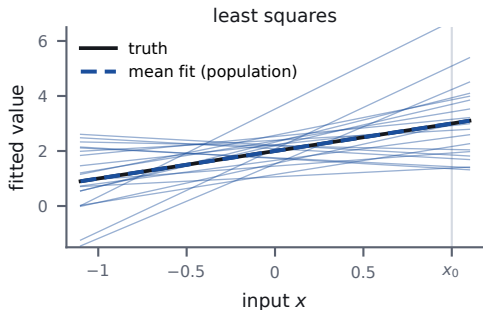
Example: Least squares vs. ridge

Same 20 training sets from before, with ridge λ set to make $a = 1/2$.



Example: Least squares vs. ridge

Same 20 training sets from before, with ridge λ set to make $a = 1/2$.



At $x_0 = 1$: least squares has **bias**² + **variance** of $0 + 4/3$, while ridge has $1/4 + 7/12$, smaller in total.